Quantum Error Correction: An Exploration at the NQCC

Monika Udo, Iha Kishore

29th June 2026 - 10th July 2026

Abstract

Quantum error correction(QEC) is a fundamental part of developing Quantum computers that have use cases further than the lab, and more so in day-to-day practice. [1] Therefore, it is important to understand and optimise these algorithms in order to be implement them. Without Quantum Error Correction, as quantum computers scale, their error rate will be far too high to perform long, complex calculations, rendering them without use. Fault-Tolerant Computers [2], are systems can continue running smoothly despite any errors caused by noise - and this is achieved by QEC.

1 Introduction

There are several different types of Quantum Error Correction Codes (QECCs) like Shor Code, Surface Code, Steane Code and Quantum Low-Density-Parity Check Codes. We chose to explore Shor code as it is one of the introductory examples of a QEC method that exposed us to the principle techniques of QEC such as parity checks, alongside the concept of stabilisers. This then allowed us to explore the more advanced and effective forms of QEC such as Surface code. Surface code is a method at the forefront of current QEC, and is a technique at the foundation of current explorations in research. [3]This prospect allowed us to develop our understanding of QEC codes: beginning with the Shor code, which was based on the classical 3-Bit Repetition Code (see 2.2.1 and 2.2.2) and has been manipulated into a QEC, to then focusing on the quantum-specific Surface Code.

On our two week project, we have both understood and implemented the Shor and Surface QEC codes, but also developed a demo to assist others, primarily non-technical, people to understand QEC through a visual representation. We have been primarily exploring error correction using quantum memory, where we are protecting an unchanging logical state against noise over time, and are not implementing logical operations that change the state of the encoded logical qubit. As a stretch goal, we further explored both Shor code and Surface code implementation over multiple logical qubits, alongside the implementation of transversal logical operations such as CNOT, X and Z.

Contents

1 Introduction **1**
1.1 Key Definitions 2
1.2 Methodology 3

2 Shor Code **3**
2.1 Background . . . 3
2.1.1 Efficiency . . . 3
2.2 Encoding . . . 4
2.2.1 3 Qubit Bit-flip Encoding . . . 4
2.2.2 3 Qubit Phase-flip Encoding . . . 4
2.2.3 9 Qubit Shor Code Encoding . . . 4
2.3 Error Injection . . . 5
2.4 Syndrome Measurement . . . 6
2.4.1 Bit Flip Syndrome . . . 7
2.4.2 Phase Flip Syndrome . . . 8
2.5 Error Correction . . . 8
2.6 Decoding . . . 9
2.7 Additional Notes . . . 9

3 Surface Code **9**
3.1 Background . . . 9
3.2 Syndrome Extraction . . . 10
3.2.1 CNOT Ordering . . . 10
3.2.2 X syndrome extraction process . . . 10
3.2.3 X syndrome phase kickback . . . 11
3.3 Error Injection . . . 11
3.4 Error Detection . . . 11
3.4.1 Classical Syndrome Extraction . . . 11
3.4.2 Hamming Weights . . . 12
3.5 Error Correction . . . 12
3.5.1 Minimum Weight Perfect Matching . . . 12
3.5.2 Correcting Along a Path . . . 13
3.6 Additional Notes . . . 13

4 In Progress - Multiple Logical Qubits **14**
4.1 Shor Code . . . 14
4.1.1 Transversal Gates . . . 14
4.2 Surface Code . . . 15
4.2.1 Transversal Gates . . . 15
4.3 Additional Notes . . . 16

5 Conclusion **16**
5.1 Further Research . . . 16
5.2 Further Development . . . 16

1.1 Key Definitions

1. $[n, k, d]$ Code - A QEC that uses n physical qubits to encode k logical qubits for a distance of d.

2. Patch - The lattice of data and ancilla qubits used to encode one logical qubit in Surface Code.

3. Stabiliser - An operator that does not change the state of a system and is used to detect errors in the system

4. Parity Check - Pairwise comparison of values to identify where an error has occured via majority voting.

5. Syndrome - The result of the parity check between a set of values, displaying whether an error has occured

1.2 Methodology

General sequence for both Error Codes:

1. Encoding of initial qubit to represent a logical qubit via data and ancilla qubits
2. Injection of Errors to demonstrate the correction algorithm
3. Syndrome Measurement for both X and Z errors
4. Error Correction
   (a) Shor Code - Blockwise correction based on parity checks
   (b) Surface Code - Correction of the nearest neighbours of ancilla qubits based off syndrome measurements
5. Decoding and Sampling

2 Shor Code

2.1 Background

The Shor Code is a 9 qubit QEC code, that requires 9 data qubits to encode the original state, alongside an additional 8 ancilla qubits for further measurements and parity checks for later error correction. Shor code is comprised of 3 three-qubit sub-blocks, where within each 3 qubit sublock by parity checks bit-flips can be detected, and by parity checks between sublocks a phase-flip on a sub-block can be detected.

Bit-flip and phase flip codes are beneficial in that through using ancilla qubits, they are non-destructive to the state of inital qubit being encoded. However, they have a large overhead, requiring 9 data qubits to encode one single qubit. Individually, neither bit flip codes or phase flip codes can detect both X and Z errors, so Shor code combines both codes to be able to detect any arbitary single-qubit error.

2.1.1 Efficiency

Shor code has been assessed to not be a viable method of QEC for current quantum computers. [4] This is because as the probability of errors increases, the probability the code fail to correctly protect the qubit increases so that eventually the probability of failing to protect the qubit is greater than the probability of an error.[4]

Shor code also lacks locality, especially with the use of CNOTs between blocks in phase-flip detection which is difficult for the hardware of a quantum computer, and can be prone to causing more errors than it fixes. [5]

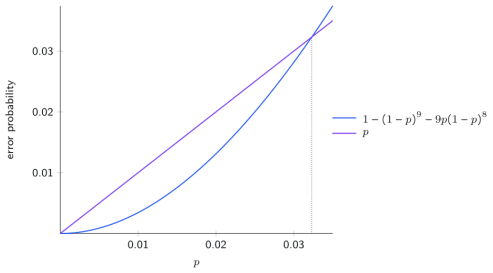


Figure 1: Shor code failure probability vs Error probability [4]

2.2 Encoding

2.2.1 3 Qubit Bit-flip Encoding

The 3 qubit bit-flip encoding uses two CNOTs between qubits 1 and 2, and qubits 1 and 3 to prepare the logical qubit representation of $\alpha\,|000\rangle + \beta\,|111\rangle$.

2.2.2 3 Qubit Phase-flip Encoding

Phase flips are not detectable in the computational basis, however, they are visible when transforming to the $|+\rangle$ and $|-\rangle$ basis. This is because $Z\,|+\rangle = |-\rangle$ and $Z\,|-\rangle = |+\rangle$, while $Z\,|0\rangle = |0\rangle$ and $Z\,|1\rangle = -\,|1\rangle$.

Therefore, the circuit for 3 qubit phase-flip encoding uses 3 Hadamard gates after the CNOTs to produce the logical qubit $\alpha\,|+++\rangle + \beta\,|---\rangle$.

2.2.3 9 Qubit Shor Code Encoding

Encoding is used to represent a single logical qubit in terms of physical data qubits. Shor code combines the encoding for both bit flip and phase flip detection together.

Firstly, the encoding circuit uses two CNOTS to encode the bit -flip repetition by converting $|0\rangle$ to $|000\rangle$ and $|1\rangle$ to $|111\rangle$, forming the state $\alpha\,|000\rangle + \beta\,|111\rangle$.

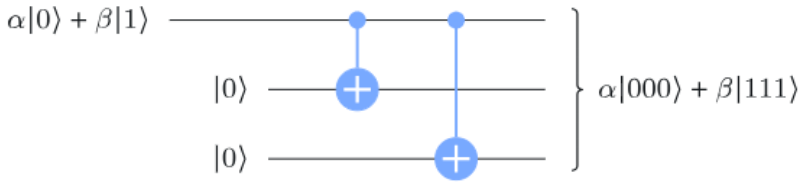


Figure 2: Encoding for one block of bit-flips[4]

Then, Hadamard gates are applied to form the phase flip encoding by transforming to the $|+\rangle$ and $|-\rangle$ basis where phase flips are directly visible. Further CNOTS are then used to again spread out and entangle to form the 9 qubit state. So we have nested the encoding for the phase flip within the bit flip.

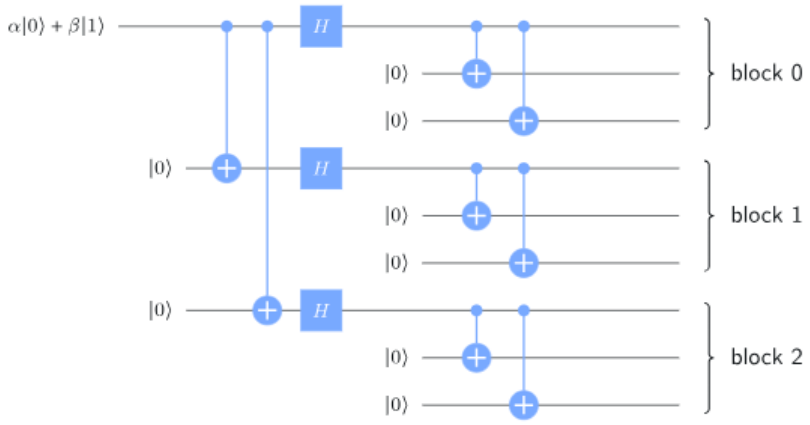


Figure 3: Complete Encoding Circuit[4]

[4]

Bit Flip Errors: Suppose the bit flip happens on the first qubit of $|\psi_L\rangle$:

$$X_1|\psi_L\rangle = a\big(|100\rangle+|011\rangle\big)\otimes\big(|000\rangle+|111\rangle\big)\otimes\big(|000\rangle+|111\rangle\big)+b\big(|100\rangle-|011\rangle\big)\otimes\big(|000\rangle-|111\rangle\big)\otimes\big(|000\rangle-|111\rangle\big)$$

To detect this error, for each group of 3 qubits, calculate the parity of every pair of qubits. For the first group, we will see an error. The parities in the other groups should be 0. [6]

Phase Flip Errors: Suppose the bit flip happens on the first qubit of $|\psi_L\rangle$:

$$Z_1|\psi_L\rangle = a\big(|000\rangle-|111\rangle\big)\otimes\big(|000\rangle+|111\rangle\big)+\big(|000\rangle+|111\rangle\big)+b\big(|000\rangle+|111\rangle\big)\otimes\big(|000\rangle-|111\rangle\big)\otimes\big(|000\rangle-|111\rangle\big)$$

We can defend against a phase error by comparing the signs of the three blocks, i.e the parity of signs. [6]

We chose to follow the standard encoding process for Shor Code as it follows the standard encoding process of implementing Shor code with 9 data qubits and 8 ancilla qubits. This encoding process can be accessed at [1]

2.3 Error Injection

Error injection with Shor code is a relatively simple process. On any of the 9 qubits we can inject at most one each of a X (bit flip) error or Z (phase flip) error.

The Shor code is a [9, 1, 3] code, which means that it uses 9 qubits with a distance of 3. The distance of an error code is the minimum number of physical qubits that must be affected to transform one valid logical state into another valid valid state.
[4] The expression $t = (d-1)/2$ is the relation between the number of errors, t, detectable and correctable with a code of distance, d. So, therefore, the Shor code can only detect at most one phase or bit flip error with 100% accuracy. If two errors occur, while Shor code can detect an error, it cannot accurately correct the error as the syndrome indistinguishably corresponds to a single error or to two errors. If three errors have occurred, this may correspond to a logical operation which is neither detectable nor correctable. When a logical operator is applied to a

state encoded in the logical qubit states, the logical operator will also produce another valid logical state, so Shor code produces a syndrome saying this other valid logical state is error free. [6]

Using the above information, we decided to only inject single errors upon our logical qubit for Shor code as this meant we could create specific testcases to ensure our Shor code implementation was 100% accurately detecting and correcting errors, without having to deal with the added difficulty of logical operations.

Theorem 1 *[Proof of $t = (d-1)/2$]*

1. ***(t errors can always be corrected)*** *This can be proven using the triangle inequality.*

Assume y is the received vector, c is the transmitted codeword, and there are t or fewer errors: $d(y, c) \leq t$.

Assume there is another valid codeword $c \neq c'$ that is also within the same distance to y. That means $d(y, c') \leq t$.

By the triangle inequality, $d(c, c') \leq d(y, c) + d(y, c')$.

So, $d(c, c') \leq 2t$.

Therefore, $t = (d-1)/2$
2. (t + 1 errors cannot always be corrected)
Assume the correct codeword c and the incorrect codeword c' sit exactly at the minimum distance from each other, meaning $d(c, c') = d$). If d is odd, we can write $d = 2t + 1$

If the channel introduces $t + 1$ errors to c, it can alter the bits in a way that moves the received word r directly toward c'.

This means that r can be t away from c and $t + 1$ away from c' so is closer to c'.

So the decoder mistakenly chooses c' as the correct codeword.

If d is even, then a similar proof applies, but r sits equally between c and c', so the correct codeword cannot be accurately retrieved.
[7]

2.4 Syndrome Measurement

To carry out syndrome measurement, we use stabilisers. Stabilisers are operators that do not change the quantum state when they are applied to error-free logical encoded qubits. They specifically map error free states back to themselves,.

Any linear combination of the logical qubits $|000\rangle$ and $|111\rangle$ is a eigenstate of the Bit flip stabiliser with a +1 eigenvalue (is unchanged by stabiliser operator) . But, if we apply this stabiliser to a state with an error, then we the state with a -1 eigenvalue. The 0 and 1 of the parity checks we do below are equivalent to the +1 and -1 the stabilizers produce as eigenvalues. The CNOT gates do not apply the unitary operation equal to the unitary operation the stabilisers represent, but instead carry out parity checks. These parity checks produce the 0 or 1 that directly correspond to the +1 and -1 eigenvalues of the stabilisers when applied to an encoded

state.

The reason the X error and Z error detection do not interfere is because the X and Z stabilisers commute (even though X and Z themselves anti-commute), so X and Z error detections can happen in any order.

2.4.1 Bit Flip Syndrome

The bit flip syndrome uses a Z_iZ_j stabiliser which acts as a parity check, measuring the parity of qubits i and j. If their parity matches, the operator produces a +1 eigenvalue in front of the operator, otherwise it produces a -1 eigenvalue.To enact the measurement of this syndrome, we use ancilla qubits and CNOTS.

Each block of three qubits has two ancilla qubits. There are two CNOTs from the first and second data qubits in that block onto one ancilla qubit. There are another two CNOTS from the second and third data qubits in that block onto another ancilla qubit.This performs the parity check as if the first and second qubit are both in state 0, we get 0 on the first ancilla qubit. If one of the qubits is in state 1, then we get 1 on the first ancilla qubit.

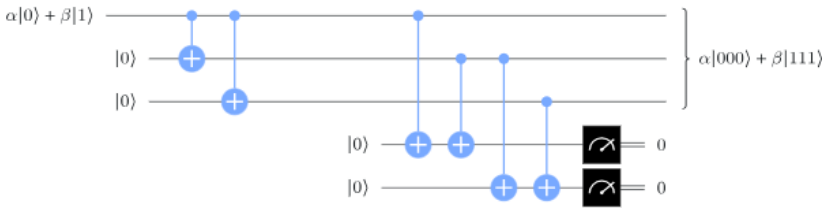


Figure 4: Syndrome measurement of the bit-flip block

Note, the qubits will never both be in state 1 as we have at most one bit flip error.

The same process occurs onto the second ancilla qubit. We can summarize this parity check as a table of ancilla qubits to produce our syndrome.

Data Qubit 1, Data Qubit 2	**Data Qubit 1, Data Qubit 3**	**Diagnosed Error State**
0	0	No Bit-Flip Error
1	0	Bit-Flip Error on Qubit 2
0	1	Bit-Flip Error on Qubit 3
1	1	Bit-Flip Error on Qubit 1

Table 1: Bit-Flip Block Parity Check Measurements

This process occurs over all three blocks.

Therefore, once these 6 ancilla qubits have been measured, we produce a 6-bit string that locates the position of the error uniquely on each of the 9 qubits.

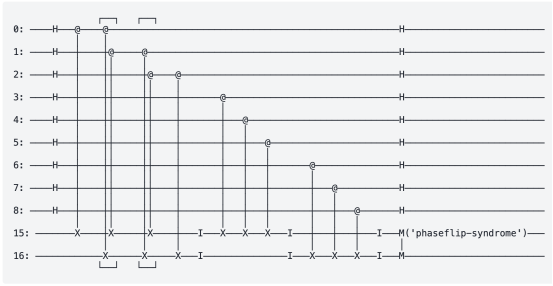


Figure 5: Phase Flip Syndrome circuit[1]

Block 1 Parity		**Block 2 Parity**		**Block 3 Parity**		
$D_1 \oplus D_2$	$D_2 \oplus D_3$	$D_4 \oplus D_5$	$D_5 \oplus D_6$	$D_7 \oplus D_8$	$D_8 \oplus D_9$	**Diagnosed Error State**
0	0	0	0	0	0	No Error
1	0	0	0	0	0	Error on Qubit 1 (D_1)
1	1	0	0	0	0	Error on Qubit 2 (D_2)
0	1	0	0	0	0	Error on Qubit 3 (D_3)
0	0	1	0	0	0	Error on Qubit 4 (D_4)
0	0	1	1	0	0	Error on Qubit 5 (D_5)
0	0	0	1	0	0	Error on Qubit 6 (D_6)
0	0	0	0	1	0	Error on Qubit 7 (D_7)
0	0	0	0	1	1	Error on Qubit 8 (D_8)
0	0	0	0	0	1	Error on Qubit 9 (D_9)

Table 2: Bit-Flip Parity Check Measurements

2.4.2 Phase Flip Syndrome

The phase flip syndrome uses a X_iX_j which also acts as a parity check, but only after we have transformed from the computational basis to the uniform superposition basis, as the X_iX_j operator measures parity in the uniform superposition basis.

To enact phase flip syndrome measurement, we use 2 ancilla qubits. Firstly, Hadamard gates are enacted upon each qubit to transform to the uniform superposition basis. This is because as HZH = X, within the uniform superposition basis, phase flips become bit flips and we can apply our method for resolving bit flips to resolving phase flips. There is a parity check of both the first block of 3 qubits and the second block of three qubits to the first ancilla qubit. There is also a parity check of both the first block of 3 qubits and the third block of 3 qubits to the second ancilla qubit.

Note: This is similar to the parity checks within each block for Bit-flips (see Table 1), howeverit does not matter which specific qubit in the block is corrected for the phase flip as it affects the block a whole.

Hadamard gates are then applied to each data qubit to transform back to the computational basis where we can then read the syndrome by measuring the phase-flip ancilla qubits.

2.5 Error Correction

Error correction is a very straightforward process for Shor code. For a bit flip, an X gate is applied to the data qubit flipped, and for a phase flip, similarly, a Z gate is applied to the qubit with the Z error.

The implementation of error correction is directly connected to the implementation of the syndromes.

2.6 Decoding

To return the initial qubit, we must decode the encoding we applied, by applying the exact inverse of the encoding procedure.

2.7 Additional Notes

We used the Qiskit Aersampler to carry out measurments of our simulation as it was simple to use, and could be further extended to adding in random noise if we later wanted to.

We encountered difficulties initially in correctly extracting syndrome measurements, but these were easily resolved after separately testing the syndrome extraction process before fully integrating syndrome extraction into the error correction process.

The inital code written was not modular so we converted the non-modular code into a modular class to increase the maintainability of our code as we further developed it, and to create seperate instances in our demo.

3 Surface Code

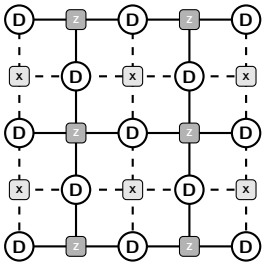
3.1 Background

The Surface Code is also a QEC code, representing one logical qubit via a lattice. It is an example of a $[[n, k, d]] = [[(d^2 + (d-1)^2, 1, d]]$ stabiliser code: $2d^2 - 2d + 1$ physical qubits are required for 1 logical qubit for a distance of d. In this implementation of a , a code with distance 3 will require 25 qubits overall and a code with distance 5 will require 81 qubits. For a distance-d code, $4d^2 - 4d + 1$ physical qubits are required.

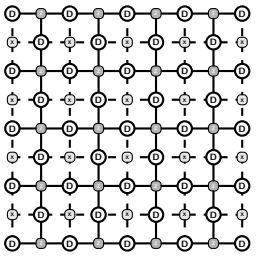
Proof For an unrotated surface code, between every pair of data qubits, there is an ancilla qubit inbetween (see Figure 2), hence for every d data qubits, there are $d-1$ qubits. Therefore, in one row / column there are $d+d-1$ qubits, and given that a surface code represents a logical qubit in a square lattice, $(2d-1)^2 = 4d^2 - 4d + 1$ qubits are used. [8]

As the distance of the Surface Code grows to reduce the possibility of errors that span boundary to boundary, the number of qubits grows quadratically, which poses significant computational overheads. An error that spans boundary-to-boundary has a Hamming weight of 0 (see 3.4.2) and thus are described as undetectable as they commute with each stabiliser the chain of errors crosses and thus no syndromes are triggered. This would then suggest there are no errors present in the patch, when infact the logical state of the system has been altered.

[H]



(a) Distance-3 lattice (25 qubits)



(b) Distance-5 lattice (81 qubits)

Figure 6: Comparison of standard unrotated surface code lattices for distances $d = 3$ and $d = 5$.

3.2 Syndrome Extraction

There are two forms of stabilisers for the Surface code, X stabilisers that form a square of 4 X gates, and Z stabilisers that form a vertex of 4 Z gates. Similarly to the use of stabilizers in Shor code, if there is no detectable error, the stabilisers will produce an eigenstate with eigenvalue +1, otherwise it will produce an eigenstate with eigenvalue -1 when applied to a given state.

With the Z stabiliser, the Z stabiliser acts as $Z_1Z_2Z_3Z_4|q_1q_2q_3q_4> = (-1)^{q_1+q_2+q_3+q_4}|q_1q_2q_3q_4>$.

This means, once again, the eigenvalue of +1 or -1 is equivalent to a parity check.

So, to get the Z syndrome measurments that check for an X error, we iterate over Z stabilisers and calculate the CNOT between the 4 adjacent data qubits to a stabiliser into an ancilla qubit.

For the X stabilisers, as they are detecting a Z error which is only noticeable in the X basis, Hadamard gates must be applied to all data qubits before a similar parity check using CNOTs is applied, and then Hadamards are further applied to transform back from the Hadamard basis to the computational basis.

We initially experienced great difficulty in correctly implementing the syndrome for both the X and Z syndrome together, and we were dealing with an issue where one syndrome would always be correct, while the other syndrome would not be generated deterministically and would instead be returning a syndrome with some given probability. We identified this error by running 1024 sample shots and examining the distribution of syndromes extracted.

We realised that due to our modular code, with previous classes being inherited into the next class, at different stages new quantum circuits were being created as different objects were instantiated. This meant that the syndrome circuit for the X and Z syndromes were separate.

As computational basis states are eigenstates of Z-stabilizers, and Hadamard basis states are eigenstates of X-stabilizer, we realised when carrying out X-syndrome exctraction, we must also apply Hadamard gates to all data qubits, as well as to the ancilla qubits.

3.2.1 CNOT Ordering

We decided to follow the Riverlane Textbook [1] code structure and also implement the greedy algorithm they use to schedule CNOTs into layers of CNOTs that could be applied in parallel.

The algorithm is implemented separately for all CNOTs that have to be implemented for all X stabilisers, and for all CNOTs that have to be implemented for all Z stabilisers.

The algorithm initially stores all the CNOTS that are necessary for a given type of stabiliser in an array. The algorithm will initialize a separate array for all CNOTs to be implemented in that layer.

This algorithm then continuously iterates over the array, and appends any CNOT whose control qubit and target qubit are not being used in other CNOTs.

3.2.2 X syndrome extraction process

We first apply a Hadamard gate to all data qubits to transform to the Hadamard basis.

3.2.3 X syndrome phase kickback

We first apply Hadamrd gates to the ancilla qubits.

When applying CNOTs to extract the X syndrome as a parity check, we do:

```
CNOT(ancilla_qubit, data_qubit)
```

Despite the reversed ordering of the CNOT with the ancilla qubit as the control and the data qubit as the target, due to phase kickback the data qubit remain unchanged while the ancilla qubit is changed.

This is because we start with the inital state: $|+\rangle_a|s\rangle_d = \frac{1}{\sqrt{2}}(|0\rangle_a|s\rangle_d + |1\rangle_a|s\rangle_d)$

As $|0\rangle_a|s\rangle_d \rightarrow |0\rangle_a|s\rangle_d$ (control 0 : untouched)

And $|1\rangle_a|s\rangle_d \rightarrow |1\rangle_a(X|s\rangle_d) = |1\rangle_a(\lambda|s\rangle_d) = \lambda|1\rangle_a|s\rangle_d$, where λ is either +1 or -1.

We then get: $\frac{1}{\sqrt{2}}(|0\rangle_a|s\rangle_d + \lambda|1\rangle_a|s\rangle_d) = \frac{1}{\sqrt{2}}(|0\rangle_a + \lambda|1\rangle_a) \otimes |s\rangle_d$.

So, the data qubit is not affected, but the ancilla qubit may or may not be flipped. [5]

3.3 Error Injection

For the project errors are injected by the user, as opposed to randomly added onto the patch and simulating noise. This in turn makes the demonstration both more engaged, but easier to understand as the user has complete control over the system and sees the effects of the error on the syndromes. Simulating noise would not add the same level of engagement, and perhaps the connections between the error they create and the response of the system would be weaker.

Errors on data qubits are added via a coordinate system: the user enters the coordinate(s) of the Pauli X, Y and Z error that should be injected onto the patch and the neighbouring syndromes then update. When an error is injected / occurs on the patch, the X and Z stabilisers anticommute, and thus produce an eigenvalue of -1 and the measurement ancillas' syndromes read 1. In an 'error-free' state, these eigenvalues are all one, and all syndromes read 0. The effect of errors can also cancel out if the combined effect commutes with the stabiliser.

3.4 Error Detection

3.4.1 Classical Syndrome Extraction

Syndromes can also be extracted classically by checking the number of data qubits that have an error around each ancilla qubit. If this number is even, i.e 0 or 2, the syndrome will read as 0 as these errors have no effect on the patch and cancel each other's erroneous effect out from the perspective of the ancilla; however if this number is odd, i.e 1 or 3, the syndrome will read as 1 and the errors will be raised. This is because all errors anticommute with the stabilisers, going from a +1 eigenvalue to a -1 eigenvalue, however, the eigenvalues produced by each error multiply together and thus when even, the eigenvalue returns to 1:

Proof: Let $n = 2m$ where $m \in \mathbb{N}$.

$$(-1)^{2m} = (-1)^2)^m$$

$$(-1)^2 = 1$$
$$1^m = 1$$

hence for even numbers, the eigenvalue is 1.

Let $n = 2m + 1$ where $m \in \mathbb{N}$.

$$(-1)^{2m+1} = ((-1)^2)^m * (-1)$$
$$(-1)^2 = 1$$
$$1^m = 1$$
$$1 * (-1)$$

hence for even numbers, the eigenvalue is -1.

3.4.2 Hamming Weights

Errors hold different weights and thus produce different number of syndromes. When an error is in the bulk of the lattice, it produces 2 syndrome measurements, one at each boundary of the error. So in a chain of errors, the -1 eigenvalues only manifest for the stabilisers at the end of the chain as the effect along the chain from each error will cancel out. An error at the boundary of the lattice will only result in one stabiliser having a -1 eigenvalue. And, an error that spans boundary to boundary has a Hamming weight of 0, so has a trivial effect on the lattice and triggers no syndromes. This is an undetectable error. If done intentionally, this is a logical operator, but if done unintentionally, this error is undetected by the stabilisers. This is a logical error and is equivalent to a bit/phase flip depending on the error type - corrupting the state of the system. However, as the distance of a patch increases, the probability of these logical errors manifesting before being corrected decreases.

3.5 Error Correction

3.5.1 Minimum Weight Perfect Matching

Per cycle of error correction, many errors may arise, and thus many syndromes are produced on the patch. As the number of syndromes on the patch grows, it becomes harder to manually pair each syndrome together in an efficient manner, keeping the total cost / number of data qubits to flip is at its minimum value. A low cost is required to reduce the number of flips / corrections made, as this corrects the errors faster, alongside reducing the risk of the gates used for corrections having errors themselves. Each syndrome can have exactly one partner in order to correct the error between them and for an unknown number of errors this 'perfect matching' becomes difficult.

Minimum Weight Perfect Matching is an optimisation algorithm that finds paths between all nodes in a graph, G. Operating on separate graphs for x and z syndromes, initially the algorithm begins with a graph where every error node is connected to all other error nodes, the algorithm then finds the distances between all nodes - in this case using the Manhattan Distance - and then calculates the set of pairings that have the lowest overall cost. After MWPM, a bipartite graph is returned, where each node has only one edge - which is shared with only one other node.

The cost of each edge is calculated by the Manhattan distance between q_1 and q_2: each qubit is stored as a set of coordinates on the lattice, hence $cost = |x_2 - x_1| + |y_2 + y_1|$. Hence, a major issue when creating our error correction algorithm was that error nodes on / near the boundary would pair to the boundary as opposed to another available error node, and thus no error would be corrected. This was overcome by adding a bias to every boundary pairing, increasing its cost by 0.5. This meant that in the event of a tie between pairing with the boundary and another error node, the pairing with the other error node would win as it has a lower cost.

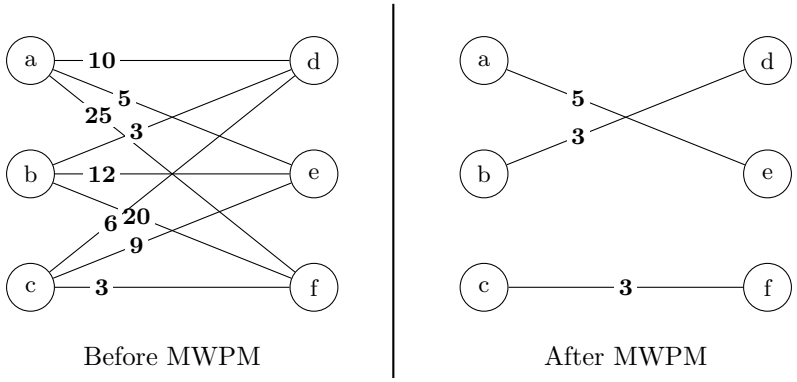


Figure 7: MWPM on errors to find the overall lowest cost pairings

Networkx Initially, I began implementing the matching algorithm myself, however, when it came to finding the optimal set of pairings from a larger graph, this became more computationally complex and I felt as though a library would implement this more efficiently, hence using networkx in the program.

3.5.2 Correcting Along a Path

To correct a bit-flip, an X gate must be applied either on the erroneous data qubit, or on another qubit adjacent to the syndrome measurement qubit to reset the syndrome to 0. Likewise, to correct a phase-flip, a Z gate must be applied either on the erroneous data qubit, or on another adjacent qubit to the syndrome measurement qubit. X syndromes indicate that a phase-flip has occured on an odd number of its neighbours, and a Z syndrome indicates that a bit-flip has occured on an odd number of its neighbours. This returns to the idea that the errors make the stabilisers anti-commute and produce a -1 eigenvalue.

After the pairs have been formed by MWPM, the corrections follow an 'L' shape, flipping each data qubit between the first ancilla qubit with a syndrome of 1, and the second ancilla qubit with a syndrome of 1 in the pair. Similar to the MWPM function, the error correction algorithm operates separately for x and z syndromes, allowing corrections on all x errors in a patch, and then all z errors in a patch.

For every data qubit in the path, the respective correction is applied so at the end of the path, the ancilla qubits that previously measured a syndrome of 1 return to 0, indicating the error has been corrected. Additionally, for bookkeeping and tracking errors, the correction affects the x, y, z error dictionaries. If the error was present in the x or z dictionary (depending on syndrome), it is discarded, and if it was not already present, it is added. If the coordinate was stored as a y error, it is discarded from that dictionary and added to the x error dictionary if the current correction cycle is for z errors and vice versa. This made it easier to track errors and debug any logical errors in our code.

3.6 Additional Notes

As mentioned briefly in 3.1 the code implemented here is an unrotated surface code, however a rotated surface code would be less recourse intensive as instead of requiring $(2d-1)^2$ qubits, it requires $2d^2-1$ qubits, which is more efficient with less overheads. [9]

The rotated surface code is a 45 degree rotation of the unrotated surface code, with boundaries that curve around the perimeter rather than the rough and smooth boundaries of the unrotated surface code. Moreover, the rotated surface code places the qubits on the vertices of

the grid squares and the stabilisers alternate inside the grid as a checkerboard pattern, whereas an unrotated surface code places qubits on the edges of the grid and the stabilisers are inside the plaquettes and on the vertices.

We chose to implement an unrotated surface code compared to a rotated surface code, as although it is far less rescourse efficient in terms of number of qubits used ($4d^2 - 4d + 1$ vs $2d^2 - 1$[8] it was easier for us to initially grasp the concept of the Surface Code and then implement it visually as a grid we easily understood.

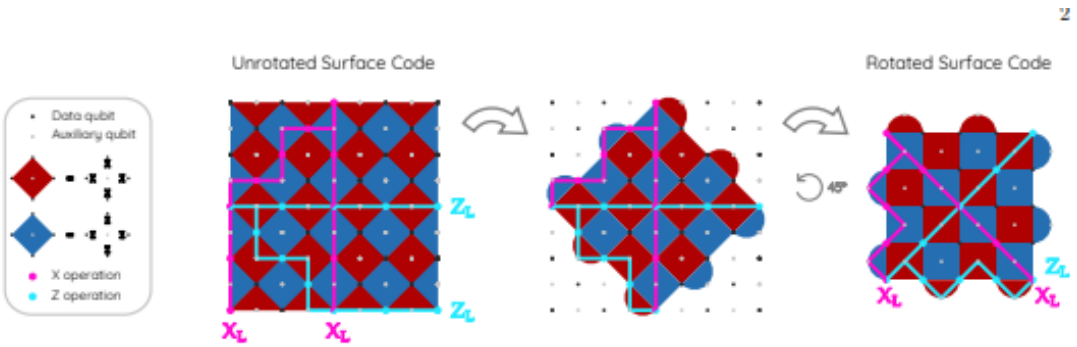


Figure 8: Transformation from unrotated to rotated surface code

4 In Progress - Multiple Logical Qubits

Eastin-Knill Theorem [10] - in summary - states that no QEC code has a universal set of gates, leading to a distinction between transversal and non transversal gates [11]. This means that to be able to do any unitary operation on an encoded qubit you have to explore implementing more non-transversal gates which can become quite complex, using magic gate distillation [12] and/or lattice surgery . A transversal gate means that a gate applied to the logical qubit is the same as applying it bit by bit to all physical qubits, as errors in one block will not propagate to another. However, non-transversal gates cannot operate in the same way as errors will propagate.

4.1 Shor Code

4.1.1 Transversal Gates

For Shor code, the X, Y, Z and the CNOT gates are transversal. However, the H, S, T gates are non-transversal. Due to limited time, we focused upon solely implementing transversal gates, but are interested in exploring the implementation of non-transversal gates as a further development within our project.

Logical X For a single logical qubit, applying the logical X gate requires applying a Z gate to one physical qubit in each of the three blocks of three qubits.

Logical Z For a single logical qubit, applying a logical Z gate is completed through applying an X gate to all 9 qubits.

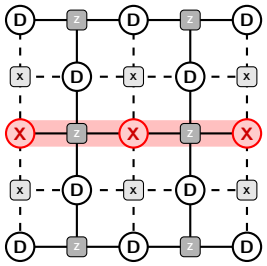
Logical CNOT To apply a CNOT gate between two logical qubits A and B where A is the control qubit and B is the target qubit, 9 CNOT gates are applied, with a CNOT gate from the ith qubit of logical qubit B to the ith qubit of logical qubit A. (So, 1 to 1, . . ., 9 to 9)

4.2 Surface Code

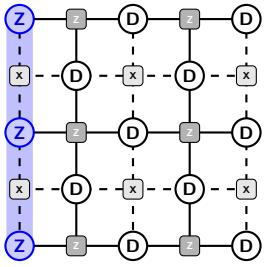
For Surface Code, the set of transversal gates include, but are not limited to, Pauli X, Y and Z gates or CNOT gates. Whereas, non-transversal gates are non-Clifford group gates like the T gates. The Clifford group contains the set of Pauli operators that will always map Pauli Gates to other Pauli Gates; for example HZH is an example of a Clifford Gate as it maps directly to a Pauli X gate.

4.2.1 Transversal Gates

Logical X In the surface code, an X gate applied to the logical qubit is equivalent to applying a logical operator across the patch. A logical operator is a chain of flips that span from one boundary of the lattice to the other on the same axis.



(a) Logical X Operator (X_L)



(b) Logical Z Operator (Z_L)

Figure 9: Logical string operators on a distance-3 planar surface code lattice with 25 total qubits.

As seen in Figure 9, a logical x operator is a chain of bit flips from the left boundary of the lattice to the right boundary and a logical z operator is a chain of phase flips from the upper boundary of the lattice to the lower boundary. Hence, a bit-flip on the logical qubit is the equivalent of applying one logical operator on the patch.

Logical Z Similarly to the bit-flip on the logical qubit, a phase-flip on the logical qubit is the application of a logical z operator on the patch.

Logical Y Since $Y = -iXZ$ applying a logical X operator followed by a logical Z operator on the patch is the equivalent of applying a Y gate on the logical qubit.

Logical CNOT A CNOT gate can be applied between two patches by executing a CNOT gate bitewise between each patch. Between two surface codes, $CNOT(A, B)$ where A is the control patch and B is the target patch, is equivalent to $CNOT(A_1, B_1), CNOT(A_2, B_2)...CNOT(A_n, B_n)$ where n is the number of data qubits in the patch.

4.3 Additional Notes

A Hadamard is a fold-transversal gate as when applied $X \leftrightarrow Z$ altering the stabilisers and boundaries of the surface code, and thus its structure. The code is only transversal when mirrored across its main diagonal. Additionally, due to time constrains, only X, Y and Z gates were implemented for multiple logical qubits for the Surface Code.

5 Conclusion

QECCs are a vital part in achieving Fault Tolerant Quantum Computing, and thus we found it crucial to both understand how this could be achieved for ourselves, but assist others in understanding too. By creating interactive demonstrations of both the Shor and Surface Codes, these tools can be used alongside academic papers and other recourses online to build layers of understanding of QEC: first via a visual explanation and then gradually building a more technical understanding through detailed explanations and mathematical theory.

This paper details our theoretical understanding and research into the mechanisms behind Shor code and Surface code while also presenting the process behind our development of implementation of these QEC codes that provide visual explanations of these codes. We have also presented the extended research we have completed into implementing QEC over multiple logical qubits where we investigated transversal and non-transversal gates and their role with Fault-Tolerant quantum computing, and then further went on to implement these gates in conjunction with the QEC codes we had previously developed.

5.1 Further Research

The primary next step in this project would be to implement non-transversal gates on the Shor and Surface codes and investigating techniques like magic state distillation to work around the limitations provided by Eastin-Knill theorem. This then allows the project to scale and implement error correction on logical qubits across a universal set of gates, especially as Gottesman-Knill Theorem asserts that quantum circuits of solely Clifford Gates can be simulated efficiently classically, the true advantage of quantum computing over classical computing lies within non-Clifford gates, which are also non-transversal. [13, 14] Therefore, time would be spent understanding the Clifford Group, and then how non-transversal and non-Clifford Gates can be implemented. Further research would also follow implementations of the rotated surface code to make our current code more efficient - using less physical qubits to encode one logical qubit. This in turn makes it easier to implement on a quantum computer, as it has lower overheads than the current unrotated surface code.

5.2 Further Development

In terms of the program itself, one key area of development would be implementing a visual representation of having multiple logical qubits being encoded and having logical gates applied to them. This could be implemented as a circuit between the logical qubits to represent the interactions on and between each logical qubit, and then a separate section to display the impact of the gate(s) on the logical qubits used for error correction.

References

[1] Riverlane. Quantum error correction. `https://www.riverlane.com/quantum-error-correction`, 2026. Accessed: 2026-07-09.

[2] Alexandru Paler and Simon J. Devitt. An introduction to fault-tolerant quantum computing. https://arxiv.org/pdf/1508.03695, 2025.

[3] Austin G. Fowler, Matteo Mariantoni, John M Martinis, and Andrew N Cleland. Surface codes: Towards practical large-scale quantum computation. https://arxiv.org/pdf/1208.0928, 2012.

[4] IBM. 9 qubit shor code. https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/correcting-quantum-errors/shor-code, 2026. Accessed: 2026-07-10.

[5] François-Marie Le Régent. Lecture 7 - foundations of quantum error correction. https://atom-tweezers-io.org/wp-content/uploads/2026/02/2026-02-$11_expqcqec2025_lecture7_qec.pdf$, 2026. $Accessed : 2026 - 07 - 10$.

[6] John Wright. Lecture 2: Shor 9-qubit code. https://people.eecs.berkeley.edu/ jswright/quantumcodingtheory24/scribe Accessed: 2026-07-10.

[7] 'manchester uni'. Parameters. bounds. https://personalpages.manchester.ac.uk/staff/yuri.bazlov/code/notes/ch2.pdf, 2026. Accessed: 2026-07-10.

[8] Riverlane. Unrotated surface codes: layout and stabilizers. https://textbook.riverlane.com/en/latest/notebooks/ch4-repcodes-to-surfcodes/layout-unrotated-surface-code-stabilizers.html, 2025. Accessed: 2026-07-06.

[9] Simon Devitt Anthony Ryan O'Rourke. Compare the pair: Rotated vs. unrotated surface codes at equal logical error rates. https://arxiv.org/pdf/2409.14765, 2007.

[10] Restrictions on transversal encoded quantum gate sets. https://arxiv.org/pdf/0811.4262, 2007.

[11] Eastin-knill theorem and its implications. https://mcgreevy.physics.ucsd.edu/f20/final-papers/2020F-220-Akhtar-Ahmed.pdf, 2020.

[12] Adam M Meier, Bryan Eastin, and Emmanuel Knill. Magic-state distillation with the four-qubit code. https://arxiv.org/pdf/1204.4221, 2024.

[13] George Biswas. On classical simulation of quantum circuits composed of clifford gates. https://arxiv.org/pdf/2405.13590, 2024.

[14] Maarten Van den Nest. Classical simulation of quantum computation, the gottesman-knill theorem, and slightly beyond. https://arxiv.org/pdf/0811.0898, 2024.